SIMATS ENGINEERING
CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I **RenugaDevi B(192311111)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
RenugaDevi B

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

1.Online Banking System – Equivalence Partitioning, Boundary Value Analysis and Random Testing

1. Introduction

An online banking system frequently fails during fund transfers, particularly at minimum and maximum transfer limits. This indicates possible defects in input validation, boundary conditions, transaction processing, or insufficient test case design

2. Critical Analysis of the Current Strategy

The existing strategy may have the following limitations:

- Insufficient testing of minimum and maximum transfer amounts.
- Excessive reliance on normal input values.
- Failure to test invalid amounts, such as zero or negative values.
- Random testing without a defined input distribution.
- Inadequate verification of account balance updates and transaction failures.

3. Application of Black-Box Testing Techniques

A. Equivalence Class Partitioning (ECP)

Assume the permitted transfer amount is ₹100 to ₹50,000, inclusive.

Equivalence class	Input amount	Expected result
Below minimum	₹50	Reject
Minimum valid class	₹100	Accept
Normal valid class	₹10,000	Accept
Maximum valid class	₹50,000	Accept
Above maximum	₹50,001	Reject
Zero amount	₹0	Reject
Negative amount	−₹100	Reject
Non-numeric input	ABC	Reject

B. Boundary Value Analysis (BVA)

For the permitted range of ₹100–₹50,000, the following values should be tested.

Test case	Amount	Expected result
TC1	₹99	Reject
TC2	₹100	Accept
TC3	₹101	Accept
TC4	₹49,999	Accept
TC5	₹50,000	Accept

C. Transaction Integrity Testing

The team should also verify that:

- The sender's balance decreases by exactly the transfer amount.
- The receiver's balance increases by the same amount.
- Failed transfers do not change either balance.
- Duplicate requests do not cause duplicate transfers.

4. Evaluation of Random Testing

Random testing should be retained as a supplementary technique, not used as the primary strategy.A random generator may rarely produce the exact minimum or maximum values. It may also generate many ordinary inputs while missing important combinations.

5. Proposed Optimized Testing Strategy

1. Define all valid and invalid input classes.
2. Apply ECP to reduce redundant test cases.
3. Apply BVA to all numeric limits.
4. Test account balance, insufficient funds, and transaction atomicity.
5. Add pairwise or risk-based combinations for amount, account type, and transfer method.

6. Conclusion

ECP and BVA provide systematic coverage of valid and invalid transfer amounts. Random testing remains useful for unexpected cases, but it cannot replace carefully selected boundary tests. Combining input testing with transaction-integrity verification significantly improves defect detection and banking reliability.

2. Healthcare Application – White-Box Testing and Test Adequacy

1. Introduction

A healthcare application has experienced critical defects after deployment despite having undergone testing. Such failures may arise from inadequate code coverage, missing control-flow paths, insufficient condition testing, or untested exception-handling logic.Since healthcare software may affect patient safety, test adequacy must be evaluated carefully.

2. Evaluation of the Existing Testing Approach

Possible weaknesses include:

- Testing only common execution scenarios.
- Achieving high statement coverage without testing both outcomes of decisions.
- Ignoring exception-handling paths.
- Failing to test combinations of patient data and clinical conditions.
- Treating code coverage as proof that the software is correct.

3. Conceptual Control Flow Graph (CFG)

Consider a simplified medication validation module.

flowchart TD

A([Start]) --> B[Read patient and medication data]

B --> C{Patient record valid?}

C -- No --> D[Report validation error]

C -- Yes --> E{Allergy detected?}

E -- Yes --> F[Block prescription and alert]

E -- No --> G{Dose within safe range?}

G -- No --> H[Reject dose and alert]

G -- Yes --> I[Save prescription]

D --> J([End])

F --> J

H --> J

I --> J

4. Coverage Analysis

A. Statement Coverage

Statement coverage requires every executable statement to run at least once.

A test that follows only the valid path might execute the prescription-saving statement but leave allergy and unsafe-dose handling untested.Therefore, statement coverage alone is inadequate.

B. Branch Coverage

Branch coverage requires both outcomes of every decision to be exercised.

The tests must cover:

- Valid and invalid patient records.
- Allergy present and allergy absent.
- Safe and unsafe medication doses.

C. Path Coverage

Path coverage examines complete execution paths through the CFG.

Representative paths include:

1. Invalid patient record → validation error.
2. Valid patient → allergy detected → prescription blocked.
3. Valid patient → no allergy → unsafe dose → rejection.
4. Valid patient → no allergy → safe dose → prescription saved.

Complete path coverage becomes impractical in larger programs containing loops, complex conditions, and many decisions. In those situations, risk-based path coverage and basis path testing are more realistic.

5. Improved Testing Strategy

- Derive tests directly from the CFG.
- Achieve 100% statement and branch coverage for the relevant module where feasible.
- Add condition coverage or MC/DC for safety-critical decisions.
- Test missing data, invalid patient records, and abnormal medication values.
- Verify that every rejection path prevents unsafe prescription storage.
- Perform integration testing with patient records, allergy databases, and pharmacy services.
- Conduct requirements-based clinical validation and regression testing.
- Use mutation testing to assess whether the tests detect deliberately introduced logic errors.

6. Justification and Conclusion

The improved strategy addresses gaps that basic functional testing may miss. CFG-based testing reveals unexecuted branches, while condition and path-oriented tests examine decision logic more deeply.

For healthcare software, structural coverage must be combined with clinical requirements validation, integration testing, and risk analysis to improve reliability and reduce patient-safety risks.

3. Retail E-Commerce System – ECP, BVA and Decision Table Testing

1. Introduction

An e-commerce system calculates order totals using price, quantity, and discount. Errors occur for specific combinations because testing each input independently does not necessarily reveal defects in the combined calculation logic.The testing strategy must therefore cover both individual input classes and interactions among inputs.

2. Limitations of the Existing Approach

The existing black-box approach may have focused on:

- Typical product prices.
- Ordinary quantities.
- Standard discounts.
- One input changing at a time.

This approach may miss errors such as a discount exceeding the order value, incorrect rounding, or applying a discount before calculating the subtotal when the business rules require a different sequence.

3. Equivalence Class Partitioning

Assume the following rules:

- Unit price must be greater than ₹0.
- Quantity must be a positive integer.
- Discount must be between 0% and 30%, inclusive.

Input	Valid class	Invalid classes
Price	Positive price	Zero, negative, non-numeric
Quantity	Positive integer	Zero, negative, fractional, non-numeric
Discount	0%–30%	Below 0%, above 30%, non-numeric

4. Boundary Value Analysis

Input	Boundary values	Expected behavior
Price	0, smallest valid price, maximum permitted price	Reject invalid values
Quantity	0, 1, maximum quantity	Enforce quantity limits
Discount	−1%, 0%, 1%, 29%, 30%, 31%	Accept only 0%–30%

5. Decision Table Testing

Consider these business rules:

- A valid order requires positive price and quantity.
- The discount must be within the allowed range.
- Valid orders receive the calculated discount.
- Invalid inputs cause rejection.

Condition/action	Rule 1	Rule 2	Rule 3	Rule 4
Price valid?	No	Yes	Yes	Yes
Quantity valid?	—	No	Yes	Yes
Discount valid?	—	—	No	Yes
Reject order?	Yes	Yes	Yes	No
Calculate total?	No	No	No	Yes

For a valid order:

$$\text{Subtotal} = \text{Price} \times \text{Quantity}$$

$$\text{Total} = \text{Subtotal} \times (1 - \frac{\text{Discount}}{100})$$

For example, if the unit price is ₹1,000, the quantity is 2, and the discount is 10%, the expected total is ₹1,800.

6. Additional Black-Box Techniques

- Pairwise testing: Covers pairs of values across price, quantity, and discount.
- Negative testing: Tests malformed and invalid inputs.
- Combinatorial testing: Covers selected higher-order combinations where interaction risk is high.
- Metamorphic testing: Checks that increasing a valid discount does not increase the total, assuming all other factors remain unchanged.
- Rounding tests: Verify totals involving fractional currency values and rounding rules.

7. Revised Test Design

The optimized suite should combine ECP and BVA for each field with decision tables for business rules. Pairwise tests can reduce the number of combinations, while additional risk-based tests cover combinations most likely to cause calculation defects.

8. Conclusion

Individual input testing is insufficient when the system's behavior depends on combinations of price, quantity, and discount. ECP, BVA, decision tables, and selected combinatorial tests provide broader and more systematic coverage without testing every possible numeric combination.

4. Complex Nested Conditions and Loops – Path Testing

1. Introduction

A software module containing nested conditions and loops has only partial path coverage. This may indicate that the team has not identified all independent paths, has constructed an incomplete CFG, or has ignored loop boundary conditions.The objective is to design a systematic path-testing strategy.

2. Analysis of the Existing Approach

Common weaknesses include:

- Missing edges in the CFG.
- Ignoring exceptional exits from loops.
- Testing only one iteration of each loop.
- Overlooking combinations of nested conditions.
- Confusing branch coverage with path coverage.

3. Constructing a CFG

Consider this pseudocode:

```
if input_valid:
   while items_remaining:
      if item_valid:
         process_item()
      else:
```

```
        record_error()
else:
    reject_input()
```

Its conceptual CFG includes:

- Input validation.
- A loop-entry decision.
- Item validation.
- Successful processing.
- Error handling.
- Loop exit and program termination.

Each decision and control transfer should be represented as a node or edge, with the loop-back edge explicitly included.

4. Cyclomatic Complexity

Cyclomatic complexity estimates the number of linearly independent paths in a connected control flow graph.

$V(G)=E-N+2P$

Where:

- E = number of edges.
- N = number of nodes.
- P = number of connected components, normally 1 for a single module.

For a single connected graph:

$V(G)=E-N+2$

The value gives the minimum number of independent basis paths needed to exercise the independent control-flow structure.

It does not equal the total number of feasible execution paths when loops are present.

5. Proposed Path-Testing Strategy

1. Construct the CFG from the source code.
2. Calculate cyclomatic complexity.
3. Identify a basis set of independent paths.
4. Design tests that execute each basis path.
5. Test loops with zero, one, two, and several iterations.
6. Test nested conditions using true and false outcomes.
7. Check loop termination and invalid-input exits.
8. Use static analysis to detect unreachable code and suspicious loop conditions.

6. Limitations and Practical Considerations

Exhaustive path coverage is generally impossible for programs with unbounded loops or many interacting conditions. Some paths may also be infeasible because the input constraints make them unreachable.

Therefore, the team should prioritize feasible basis paths, loop boundaries, error-handling paths, and high-risk combinations.

7. Conclusion

Path testing supported by a correctly constructed CFG and cyclomatic complexity provides a disciplined method for selecting independent execution paths. Loop-boundary tests and termination checks are essential for exposing defects that branch coverage alone may miss.

5. Financial Application – Hybrid Black-Box and White-Box Testing

1. Introduction

A financial application uses both black-box and white-box testing, yet production defects continue to occur. This suggests that the techniques may have been applied independently without adequate integration, or that test adequacy criteria were poorly selected.Financial applications require accurate calculations, transaction integrity, and strict enforcement of business rules.

2. Critical Evaluation

Possible gaps include:

- Black-box tests that cover ordinary inputs but miss boundaries.
- White-box tests that execute code without verifying financial correctness.
- High statement coverage but incomplete decision outcomes.
- Missing exception and rollback paths.
- Insufficient integration testing.
- No systematic traceability between requirements and test cases.

3. Improved Black-Box Testing

Equivalence Partitioning

Partition financial inputs into valid and invalid classes.

For example, if an account permits transfers from ₹100 to ₹50,000:

- Below ₹100: invalid.
- ₹100–₹50,000: valid.
- Above ₹50,000: invalid.
- Zero and negative amounts: invalid.

Boundary Value Analysis

Test values immediately below, at, and above each limit. Include decimal precision, rounding, and currency conversion boundaries where applicable.

Decision Tables

Use decision tables for combinations of:

- Account status.
- Available balance.
- Transfer amount.
- Transaction authorization.
- Daily transfer limit.

These tests verify that the expected business action is selected for each meaningful combination.

4. Improved White-Box Testing

- Construct CFGs for transaction validation and settlement.
- Achieve statement and branch coverage for critical modules.
- Use basis path testing to exercise independent control-flow paths.
- Apply MC/DC to safety- or security-critical compound decisions where appropriate.
- Test exception handling, retries, rollback, and timeout paths.
- Use static analysis and data-flow analysis to identify risky code.

5. Integration of Both Approaches

Testing layer	Primary technique	Purpose
Input validation	ECP and BVA	Detect invalid and boundary inputs
Business rules	Decision tables	Detect interaction defects
Program logic	CFG and branch testing	Detect untested decision outcomes
Independent paths	Basis path testing	Exercise distinct control-flow behavior
Transaction processing	Integration testing	Verify atomicity and consistency
Regression	Automated test suite	Prevent previously fixed defects from returning

6. Adequacy Criteria

The team should measure:

- Requirements coverage.
- Statement and branch coverage.
- Independent path coverage.
- Condition or MC/DC coverage where justified.
- Boundary-value coverage.
- Mutation-test effectiveness.
- Defect escape rate and regression effectiveness.

7. Conclusion

A hybrid strategy is more effective when black-box and white-box techniques complement one another. Black-box testing verifies externally observable behavior, while white-box testing exposes untested control flow. Their integration with risk-based prioritization, transaction-integrity testing, and regression automation reduces production defects more effectively than isolated testing techniques.

6. HR Payroll System – Bonus Calculation at 5, 10 and 15 Years

1. Introduction

A specialized HR payroll system calculates bonuses using years of service and performance ratings. Employees at exactly 5, 10, and 15 years receive incorrect payouts.The concentration of defects at these values strongly suggests boundary-related errors in the bonus calculation logic.

2. Critical Analysis

Potential causes include:

- Incorrect use of > and >= operators.
- Overlapping or missing service-duration ranges.
- Incorrect handling of anniversary dates.
- Rounding errors in bonus percentages.
- Incorrect interaction between service duration and performance ratings.

3. Equivalence Class Partitioning

Assume, for illustration, that the bonus tiers are:

- Less than 5 years.
- 5 to less than 10 years.
- 10 to less than 15 years.
- 15 years or more.

Class	Years of service	Expected tier
EC1	4 years	Tier 1
EC2	5–9 years	Tier 2
EC3	10–14 years	Tier 3
EC4	15 years or more	Tier 4

4. Boundary Value Analysis

If service duration is measured in completed whole years, the key test values are:

Boundary	Values to test
5 years	4, 5, 6
10 years	9, 10, 11
15 years	14, 15, 16

5. Additional Test Cases

- Minimum and maximum performance ratings.
- Invalid or missing ratings.
- Employees with incomplete years of service.
- Leap-day joining dates.
- Employees whose anniversary falls on a payroll cutoff date.
- Changes in performance rating near a service threshold.
- Rounding and currency precision.

6. Why BVA and ECP Are Suitable

ECP systematically covers different service-duration categories. BVA concentrates tests around the exact thresholds where the reported failures occur.

Exploratory testing remains valuable for uncovering unexpected business-rule defects, but BVA and ECP provide repeatable, auditable coverage for financial calculations.

7. Conclusion

The redesigned suite should prioritize the 5-, 10-, and 15-year boundaries, validate all service-duration classes, and combine each with relevant performance ratings. This approach provides systematic evidence that bonus thresholds are implemented correctly and reduces the risk of incorrect payroll payouts.

7. Laboratory Temperature Control – CFG and MC/DC

1. Introduction

A laboratory temperature controller uses nested if-else conditions to operate heaters, coolers, and alarms. A specific sensor-malfunction sequence failed to trigger an alarm after deployment.Because the system controls laboratory equipment, missed alarms can create safety hazards. Testing must therefore verify decision logic and abnormal sensor conditions.

2. Conceptual Control Flow Graph

Consider this illustrative logic:

```
Read temperature and sensor status
if sensor_fault:
    disable_heater()
    disable_cooler()
    activate_alarm()
else:
    if temperature < lower_limit:
        activate_heater()
    else if temperature > upper_limit:
        activate_cooler()
    else:
        maintain_temperature()
```

The corresponding conceptual CFG is:

```
flowchart TD
    A([Start]) --> B[Read sensor data]
    B --> C{Sensor fault?}
    C -- Yes --> D[Disable heater and cooler]
    D --> E[Activate alarm]
    C -- No --> F{Temperature below lower limit?}
    F -- Yes --> G[Activate heater]
    F -- No --> H{Temperature above upper limit?}
```

H -- Yes --> I[Activate cooler]

H -- No --> J[Maintain temperature]

E --> K([End])

G --> K

I --> K

J --> K

3. Why Branch Coverage May Be Insufficient

Branch coverage requires both true and false outcomes of each decision to be executed. However, it does not necessarily demonstrate that each individual condition in a compound Boolean expression independently affects the decision.

For example:

alarm = sensor_fault OR temperature_critical

A test with sensor_fault = true may activate the alarm regardless of the temperature condition. This does not prove that the temperature condition independently influences the result.

4. MC/DC Testing

Modified Condition/Decision Coverage (MC/DC) requires each condition in a compound decision to be shown to independently affect the decision outcome, while the other conditions are held appropriately fixed.

For the decision:

$D=A \vee B$

where

A is sensor fault and

B is critical temperature, an illustrative MC/DC test set is:

Test	A	B	Alarm D
T1	False	False	False
T2	True	False	True
T3	False	True	True

- T1 versus T2 demonstrates the independent effect of A
- T1 versus T3 demonstrates the independent effect of B

5. Improved Testing Strategy

1. Construct a CFG from the real source code.
2. Test both outcomes of every decision.
3. Design MC/DC tests for critical compound decisions.
4. Inject sensor faults, missing readings, stale data, and out-of-range measurements.
5. Test simultaneous heater and cooler requests to ensure unsafe activation cannot occur.
6. Verify alarms, safe shutdown, and fault recovery.
7. Test timing constraints and hardware interfaces.
8. Use independent requirements-based safety tests and regression testing.

6. Justification

MC/DC is more rigorous than statement or branch coverage for compound decisions. It helps expose faults where one condition masks another.Nevertheless, 100% MC/DC alone does not guarantee safety. Timing, hardware failures, requirements completeness, and fault recovery must also be verified.

7. Conclusion

A CFG-based white-box strategy supported by branch coverage and MC/DC is appropriate for the temperature controller. Combined with sensor-fault injection and hardware-in-the-loop testing, it improves the likelihood that critical alarms activate correctly under abnormal operating conditions.

8. Travel Booking Portal – Decision Tables and Cause-Effect Testing

1. Introduction

A travel booking portal permits users to apply multiple promotional codes. Certain combinations produce a negative total amount due, indicating that promotional rules may interact incorrectly.Testing each promotional code separately is insufficient because the failure depends on combinations of price, promo type, and user tier.

2. Limitations of Isolated Testing

The existing approach may have tested:

- Each promotional code individually.
- Standard booking prices.
- Only one user tier.
- Ordinary discount combinations.

3. Equivalence Classes

Input	Valid classes	Invalid classes
Booking price	Positive permitted amount	Zero, negative, malformed
Promo type	Valid, stackable or exclusive promo	Unknown, expired, incompatible promo
User tier	Free, Basic, Premium	Unknown or invalid tier

4. Decision Table

Consider a simplified policy:

- A booking must have a positive price.
- A promotional code must be valid.
- Some promotions are restricted to Premium users.
- Discounts cannot reduce the total below zero.

Condition/action	Rule 1	Rule 2	Rule 3	Rule 4
Price positive?	No	Yes	Yes	Yes
Promo valid?	—	No	Yes	Yes
User eligible?	—	—	No	Yes
Reject request?	Yes	Yes	Yes	No
Apply discount?	No	No	No	Yes
Total below zero?	No	No	No	Must prevent

5. Additional Combination Tests

- Two compatible promotional codes.
- Two mutually exclusive codes.
- The same code applied twice.
- Expired and valid codes used together.
- Premium-only code used by a Free user.
- Discounts equal to the booking price.

6. Optimized Test Selection

Use ECP and BVA for individual fields, then apply decision-table testing to the business rules. Pairwise testing can cover interactions among price category, promo type, and user tier without testing every possible combination.High-risk combinations, particularly stacked discounts and incompatible promotions, should receive dedicated tests even if they are not selected by a pairwise suite.

7. **Correctness Properties**

For every valid booking:

Total Amount Due≥0

The final total should also match the documented discount order, eligibility rules, rounding policy, and tax rules.

8. Conclusion

Decision tables reveal interactions that isolated input testing can miss. By combining them with ECP, BVA, and targeted combinatorial testing, the portal can reduce redundant cases while increasing coverage of promotional conflicts and preventing negative totals.

9. Legacy Database Migration – Loop Testing and Cyclomatic Complexity

1. Introduction

A legacy database migration tool validates records through deeply nested loops. The team reports high statement coverage, but infinite-loop crashes continue to occur in production.This indicates that statement coverage has not adequately tested loop termination and control-flow behavior.

2. Why Statement Coverage Is Insufficient

Statement coverage proves that executable statements have run at least once. It does not prove that:

- Loop conditions eventually become false.
- Every loop executes the correct number of iterations.
- Nested loops terminate for all supported inputs.
- Error conditions allow loops to exit.
- Loop variables are updated correctly.

3. Conceptual CFG

Consider:

text

```
for each batch:
    index = 0
    while index < batch_size:
        validate_record(index)
```

```
    if record_invalid:
        log_error()
    index = index + 1
```

The CFG must include:

- Outer-loop entry and exit.
- Inner-loop condition.
- Record validation.
- Error-handling branch.
- Index update.
- Back edges for both loops.

A missing or incorrectly implemented index update can cause the inner loop to continue indefinitely.

4. Cyclomatic Complexity

For a connected CFG:

$V(G)=E-N+2$

Alternatively, under the usual assumptions for a structured program, cyclomatic complexity can be calculated as one plus the number of binary decision points.

For example, a module with four binary decision points has a cyclomatic complexity of approximately 5, assuming the standard structured-flow assumptions hold.

The value helps identify a minimum basis set of independent paths, but it does not directly count all possible executions of nested loops.

5. Proposed Loop and Path Testing Strategy

Test the inner loop with:

- Zero records.
- One record.
- Two records.
- The maximum permitted batch size.
- A batch containing invalid records.
- A record that triggers an exception.
- A batch that reaches an error-recovery path.

Test the outer loop with:

- Zero batches.
- One batch.
- Multiple batches.
- An empty batch between non-empty batches.
- Failure in one batch followed by successful processing of another.

Also verify that every loop variable progresses toward termination and that exceptions cannot bypass necessary updates or exit conditions.

6. Additional Techniques

- Static analysis to detect non-terminating loops.
- Timeouts and watchdogs in test environments.
- Property-based tests using varying batch sizes.
- Data-flow analysis for loop-control variables.
- Mutation testing that deliberately removes an increment or changes a loop condition.
- Regression tests for each discovered termination defect.

7. Conclusion

High statement coverage does not demonstrate loop correctness. CFG-based basis path testing, cyclomatic complexity analysis, and explicit loop-boundary tests are necessary to detect non-termination defects. These methods improve confidence in the migration tool's ability to process empty, small, large, and malformed datasets reliably.

10. Food Delivery App – State Transition Testing

1. Introduction

A food delivery application tracks orders through the states Ordered, Preparing, Out for Delivery, and Delivered. Users report that the tracker becomes stuck when a driver cancels during delivery.This problem concerns dynamic state transitions rather than only individual input values.

2. Limitations of ECP and BVA

ECP and BVA are useful for input fields such as delivery distance, order amount, and estimated delivery time. However, they do not adequately cover sequences of events that change the application's state.For example, a driver cancellation occurring after dispatch may require a different response from a cancellation before food preparation.

3. State Transition Model

The intended state transitions can be represented as follows:

stateDiagram-v2

[*] --> Ordered

Ordered --> Preparing: Restaurant accepts

Ordered --> Cancelled: Order cancelled

Preparing --> OutForDelivery: Food dispatched

Preparing --> Cancelled: Cancellation permitted

OutForDelivery --> Delivered: Successful delivery

OutForDelivery --> ReassignmentPending: Driver cancels

ReassignmentPending --> OutForDelivery: New driver assigned

ReassignmentPending --> Cancelled: Delivery cannot continue

Delivered --> [*]

Cancelled --> [*]

4. State Transition Test Cases

Test	Initial state	Event	Expected state
TC1	Ordered	Restaurant accepts	Preparing
TC2	Preparing	Food dispatched	Out for Delivery
TC3	Out for Delivery	Driver cancels	Reassignment Pending
TC4	Reassignment Pending	New driver assigned	Out for Delivery
TC5	Reassignment Pending	Reassignment fails	Cancelled or defined recovery state
TC6	Delivered	Driver cancellation event	No state change; event rejected or ignored
TC7	Cancelled	Delivery confirmation arrives late	No invalid transition
TC8	Out for Delivery	Duplicate dispatch event	No duplicate transition

5. State and Transition Coverage

- State coverage: Visit every defined state.
- Transition coverage: Exercise every valid transition.
- Invalid-transition testing: Attempt events that must not be permitted.
- Transition-sequence testing: Test meaningful sequences, especially cancellation and reassignment.
- Event-order testing: Verify that delayed or duplicate events do not corrupt the state.

6. Improved Testing Strategy

1. Create a complete state transition diagram.
2. Define valid and invalid transitions.
3. Test every state and transition.
4. Test driver cancellation at every relevant delivery stage.
5. Simulate network delays and duplicate notifications.
6. Verify backend state and mobile UI consistency.
7. Test recovery when driver reassignment fails.
8. Add regression tests for every stuck-state defect.

7. Conclusion

State transition testing directly addresses the defect because the application behavior depends on the sequence of events and current state. Combining state coverage with transition-sequence testing and ECP/BVA for individual fields produces a more complete strategy for reliable real-time tracking.

11. Cybersecurity Authentication – Hybrid Testing and Brute-Force Prevention

1. Introduction

A cybersecurity authentication module uses password strength, OTP timing, and IP location to determine whether access should be granted. Brute-force vulnerabilities were detected despite previous testing.This suggests that tests focused on individual authentication functions may have missed weaknesses in repeated attempts, rate limiting, or interactions between security controls.

2. Critical Assessment

Possible weaknesses include:

- Testing only successful and unsuccessful logins.
- Failing to test repeated authentication attempts.
- Ignoring OTP expiration and replay.
- Testing password strength without verifying rate limiting.
- Missing branches for unusual IP changes.
- Failing to test recovery and account-lockout behavior.

3. Black-Box Security Testing

Equivalence Partitioning

Partition inputs into:

- Valid and invalid passwords.
- Correct, incorrect, expired, and reused OTPs.
- Trusted, unfamiliar, and blocked IP locations.
- Permitted and excessive authentication attempts.

Boundary Value Analysis

Test:

- OTP immediately before, at, and after expiration.
- The last permitted failed login and the next attempt.
- Passwords at minimum length and strength thresholds.
- Rate-limit windows immediately before and after reset.

Error Guessing

Use security expertise to target likely failure points, such as OTP reuse, inconsistent rate limiting across sessions, and attempts to bypass one factor after another factor fails.

4. White-Box Path Analysis

Construct a CFG for the authentication process:

1. Receive login request.
2. Validate password.
3. Check account status and rate limits.
4. Validate OTP.
5. Evaluate IP-based risk.
6. Grant, reject, or challenge the request.
7. Update failed-attempt counters and audit records.

5. Improved Hybrid Strategy

- Apply ECP and BVA to password, OTP, and rate-limit inputs.
- Use decision tables for combinations of password validity, OTP validity, account status, and IP risk.
- Use CFG-based branch and basis path testing.
- Apply error guessing and threat modeling to identify likely attack paths.
- Perform controlled security testing for brute-force resistance.
- Verify rate limiting across sessions and distributed requests.

6. Justification

Black-box testing validates externally observable security behavior. White-box testing reveals missing checks and incorrect control-flow decisions. Error guessing adds adversarial scenarios that may not appear in ordinary requirements-based tests.However, code coverage alone cannot demonstrate resistance to brute-force attacks. Security testing must also assess rate limits, abuse resistance, monitoring, and the effectiveness of defensive controls.

7. Conclusion

A hybrid strategy combining ECP, BVA, decision tables, CFG-based path analysis, and adversarial testing offers stronger protection than ordinary functional testing. It helps identify weaknesses in authentication logic and improves the module's security posture.

12. E-Learning Platform – Subscription Expiration and Equivalence Partitioning

1. Introduction

An e-learning platform offers Free, Basic, and Premium access based on subscription dates and course types. Defects occur when users upgrade exactly on the expiration date.This suggests ambiguity or incorrect implementation of date-boundary rules and subscription transitions.

2. Evaluation of Existing Testing

Possible weaknesses include:

- Testing upgrades only several days before expiration.
- Testing expiration only after the subscription ends.
- Ignoring the exact expiration timestamp.
- Failing to test time zones.
- Treating all course types as having identical access rules.
- Testing subscription types separately without examining upgrade combinations.

4. Subscription-Type Partitioning

Test the following combinations:

- Free user accessing a Free course.
- Free user attempting a Basic course.
- Basic user accessing a Basic course.
- Basic user attempting a Premium course.
- Premium user accessing a Premium course.
- Basic-to-Premium upgrade near expiration.
- Premium-to-Basic downgrade at renewal.
- Expired subscription attempting access.

5. Boundary Value Analysis

For each important expiration time, test:

- Immediately before expiration.
- Exactly at expiration.
- Immediately after expiration.

Where timestamps are stored to the second, use the adjacent representable timestamps supported by the system.Also test date changes across midnight, daylight-saving transitions where relevant, and consistent treatment of UTC versus local time.

6. Improved Test Design

1. Define the subscription lifecycle and access rules.
2. Partition dates into active, boundary, and expired classes.
3. Partition subscription types and course categories.
4. Use decision tables for subscription type, course type, and time status.

5. Test upgrade and downgrade sequences.
6. Verify consistency between the UI, authorization service, and billing system.
7. Automate boundary tests using a controllable test clock.
8. Include regression tests for renewal and expiration defects.

7. Conclusion

ECP helps cover distinct date and subscription classes, while BVA targets the exact expiration point. Decision tables and transition tests add coverage for interactions between course access, subscription changes, and timing. Together, these techniques reduce access-control errors and provide a structured approach to testing subscription transitions.

13. Smart-Home Lighting Controller – Branch and Condition Coverage

1. Introduction

A smart-home lighting controller adjusts brightness using ambient-light sensors. The source code contains switch-case statements and complex Boolean expressions.Relying only on statement coverage may leave decision outcomes and individual Boolean conditions insufficiently tested, increasing the risk of unexpected behavior when the software interacts with hardware.

2. Limitations of Statement Coverage

Statement coverage confirms that executable statements have run. It does not guarantee that:

- Every switch-case alternative has been selected.
- Both outcomes of every Boolean decision have occurred.
- Each condition has independently influenced the result.
- Sensor faults or invalid values have been handled.
- The hardware receives the correct brightness command.

3. Example Control Logic

Consider:

text

```
if sensor_valid AND occupancy_detected:
    if ambient_light < threshold:
        brightness = HIGH
    else:
        brightness = LOW
else:
    brightness = SAFE_DEFAULT


switch operating_mode:
```

case AUTO: apply brightness

case MANUAL: apply manual setting

case NIGHT: apply night setting

default: reject invalid mode

4. Branch and Condition Coverage

Branch Coverage

Test both outcomes of:

- sensor_valid AND occupancy_detected.
- ambient_light < threshold.
- Every switch-case alternative and the default branch, where applicable.

Condition Coverage

Test each atomic condition as both true and false.

For the compound condition, the tests must separately exercise:

- sensor_valid = true and false.
- occupancy_detected = true and false.

Condition coverage is not equivalent to MC/DC; the latter additionally requires demonstrating each condition's independent effect on the overall decision.

5. Improved White-Box Strategy

1. Construct a CFG for the complete brightness-control module.
2. Enumerate every switch-case outcome.
3. Achieve full branch coverage for the relevant module.
4. Achieve condition coverage for Boolean expressions.
5. Apply MC/DC where decision complexity and safety risk justify it.

6. Justification

Branch coverage exposes untested decision outcomes. Condition coverage exposes atomic Boolean conditions that may otherwise remain false or true in all executed tests. Hardware-in-the-loop tests verify that correct software decisions translate into correct physical behavior.

7. Conclusion

Statement coverage alone is inadequate for the lighting controller. A combination of CFG-based branch coverage, condition coverage, targeted MC/DC, and hardware-integration testing provides stronger evidence that sensor conditions and operating modes are handled correctly.

14. Logistics Management – Robust Boundary Value Analysis

1. Introduction

A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors involving zero or negative weights.This indicates inadequate input validation and insufficient testing of invalid numeric boundaries.

2. Analysis of the Failure

Possible causes include:

- Testing only positive weights.
- Assuming users always enter valid data.
- Missing explicit rejection of zero or negative weights.
- Incorrect comparison operators.
- Incorrect handling of decimal values and unit conversions.
- Failing to verify error handling before the shipping-cost calculation.

Negative values may produce negative charges, while zero weights may bypass calculation branches unexpectedly.

3. Robust Boundary Value Analysis

Assume the minimum valid shipment weight is 0.1 kg and the maximum supported weight is 100 kg.

Test	Weight	Expected result
TC1	−0.1 kg	Reject
TC2	0 kg	Reject
TC3	0.1 kg	Accept
TC4	0.2 kg	Accept
TC5	99.9 kg	Accept
TC6	100 kg	Accept
TC7	100.1 kg	Reject

4. Equivalence Partitioning

Additional classes include:

- Valid positive weights.
- Zero weights.
- Negative weights.
- Weights above the supported maximum.
- Non-numeric values.
- Missing weights.
- Invalid units, such as an unsupported measurement unit.

5. Additional Test Cases

- Small parcel with express delivery.
- Maximum-weight parcel with standard delivery.
- Invalid destination zone.
- Unsupported delivery speed.
- Decimal weights near rounding boundaries.
- Unit conversion between kilograms and grams.
- Invalid weight combined with an otherwise valid destination and speed.

6. Improved Testing Strategy

1. Define the permitted weight range and units.
2. Apply robust BVA to minimum and maximum values.
3. Apply ECP to numeric, missing, malformed, and unsupported-unit inputs.
4. Use decision tables for weight, zone, and delivery speed.
5. Verify invalid inputs are rejected before cost calculation.
6. Check that valid shipping charges are non-negative and consistent with the pricing policy.
7. Automate regression tests for all invalid-input defects.

7. Conclusion

Robust BVA systematically examines values outside the permitted range and is particularly suitable for detecting zero and negative-weight defects. Combined with ECP and decision tables, it improves validation coverage and prevents invalid inputs from corrupting shipping-cost calculations.

15. Medical Imaging Software – Basis Path Testing and Prime-Sized Datasets

1. Introduction

Medical imaging software processes large datasets through parallel loops. Inconsistent results occur only when the data size is a prime number, while the testing team relies heavily on random testing.This pattern suggests a possible defect in partitioning, parallel scheduling, array indexing, chunk allocation, or remainder handling.

2. Critical Evaluation of Random Testing

Random testing is useful for exploring large input spaces and discovering unexpected failures. However, it may not reliably generate:

- Prime-number dataset sizes.
- Values immediately around parallel chunk boundaries.
- Sizes that leave one worker with a remainder element.
- Specific thread-interleaving sequences.
- Minimal datasets that expose initialization defects.

Random testing alone is therefore inadequate for a specialized mathematical module with a reproducible input pattern.

3. White-Box Analysis

The team should inspect the source code for:

- Array-boundary calculations.
- Division of data into parallel chunks.
- Remainder handling.
- Loop initialization and termination.
- Shared variables and synchronization.
- Thread-dependent updates.
- Floating-point reduction order.

For example, if a dataset of n elements is divided among *k* workers, the implementation must correctly handle both the quotient and remainder when *n* is not evenly divisible by k

4. Basis Path Testing

Construct a CFG for the processing algorithm, including:

1. Input validation.
2. Dataset initialization.
3. Worker or chunk allocation.
4. Parallel processing loop.
5. Remainder handling.
6. Result aggregation.
7. Error and completion paths.

Calculate cyclomatic complexity:

V(G)=E−N+2P

For a single connected CFG:

V(G)=E−N+2

5. Structured Test Data

Use deterministic datasets with sizes such as:

Dataset category	Example sizes	Purpose
Empty or minimum	0, 1	Validate initialization and minimum-size handling
Small prime sizes	2, 3, 5, 7	Expose small partitioning defects
Composite sizes	4, 6, 8, 12	Compare against ordinary chunking
Chunk boundaries	k−1,*k*,k+1	Test worker allocation
Larger prime sizes	101, 997	Test remainder handling at scale
Large datasets	Requirement-defined maximum	Test memory and stability

Here,k represents the number of elements or workers relevant to a particular partitioning boundary. The actual test sizes should be chosen from the implementation's chunking algorithm.

6. Mathematical and Concurrency Verification

- Compare results with a trusted sequential reference implementation.
- Verify invariants such as element counts and output dimensions.
- Repeat identical inputs across different thread counts.
- Test with different scheduling configurations.
- Check deterministic numerical tolerances where floating-point operations are involved.
- Use race-detection and memory-safety tools where available.
- Add regression tests for every prime-sized failure.
- Retain randomized stress testing with fixed seeds for reproducibility.

7. Justification

Basis path testing targets independent control-flow behavior, while deterministic prime-sized inputs target the specific mathematical and partitioning conditions associated with the observed defect. Comparing parallel results against a trusted sequential implementation provides an independent correctness oracle. Because the software is used for medical imaging, mathematical verification, data-integrity checks, and risk-based validation should accompany structural coverage.

8. Conclusion

Random testing should be retained as a supplementary stress-testing technique, but it should not be the primary strategy. A stronger approach combines CFG construction, basis path testing, prime and boundary-sized datasets, parallel execution tests, and comparison with a trusted reference implementation. This combination improves defect detection, numerical consistency, and system stability.